# Noah Backend Guide

## Architecture & Project Structure

Version 0.1 | PAYTO Technologies UG | Author: Swapnanil Chakraborty

## 1 Introduction

### 1.1 What is Noah?

Noah is the AI assistant embedded in the PAYTO ecosystem. Rather than a simple chatbot, it is built as an **Agentic AI System**: it reasons over requests, pulls verified data from PAYTO's own sources, triggers application actions, and replies in natural conversational language.

The core idea is to fuse an LLM's reasoning with the predictability of deterministic application logic:

`Noah` = `LLM reasoning` + `Tool execution` + `PAYTO data`

Think of it as ChatGPT's language ability, Siri's ability to act, and PAYTO's merchant, loyalty, and commerce data, combined. The FastAPI backend is the assistant's "brain"; the Flutter app is purely its face.

### 1.2 Project Goals

**Conversational intelligence.** Users talk naturally by text or voice — "Show me my barcode," "Where can I buy frozen pizza under €2?"

**Context awareness.** Follow-up requests should resolve against prior turns. If a user asks for nearby cafés and then says "only the ones under €5," Noah must know what "the ones" refers to.

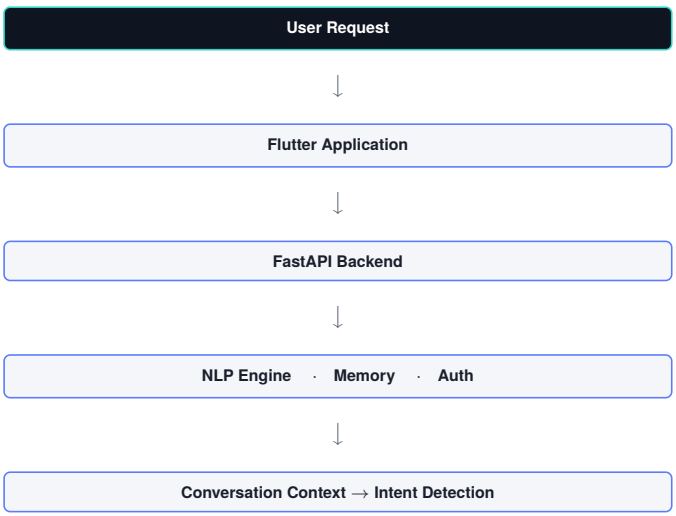
**Knowledge grounding.** Noah retrieves facts from PAYTO's own databases before answering, instead of relying on the model's internal knowledge — reducing hallucination and keeping answers current. Sources include the merchant database, reward policies, FAQs, offer catalogues, product data, and (later) a recommendation engine.

**Tool execution.** Some requests demand an action, not prose: opening the barcode screen, searching merchants, launching Maps navigation, showing a profile, or pulling purchase history. Noah invokes a dedicated tool instead of describing the action in words.

**Scalability.** The backend must absorb future capabilities — recommendations, reinforcement learning, voice, image understanding, OCR, calendars, bookings, multi-agent workflows — without a redesign.

## 2 System Overview

Unlike a conventional REST service, every request passes through several reasoning stages before a reply is generated.




Note that the LLM is not the whole system — it is one component that sits alongside the planner, retrieval engine, memory, and authentication layers.

## 3 Backend Philosophy

The guiding principle is a strict **separation of reasoning and execution**. The LLM never queries Firestore, calls Google Maps, retrieves merchants, searches nearby stores, edits databases, or authenticates users. It only decides *what* should happen; dedicated Python modules decide *how*.

This split improves maintainability, debuggability, security, reliability, and explainability.

> **Example.** User: "Show me my loyalty card."
> LLM resolves intent → `OPEN_BARCODE`
> Backend calls `BarcodeTool` → returns `{"action":"open_barcode"}`
> Flutter receives the JSON and navigates to the Barcode screen.

The LLM never talks to Flutter directly — everything passes through structured JSON.

## 4 Project Structure

The backend is an independent FastAPI project with one responsibility per directory:

```
noah-backend/
app/
api/       # routes & I/O
core/      # config, security
models/    # pydantic schemas
services/  # external integrations
tools/     # executable actions
planner/   # request dispatcher
rag/       # retrieval pipeline
llm/       # model interface
nlp/       # language understanding
memory/    # conversation state
prompts/   # prompt templates
config/
utils/
tests/  docs/  scripts/
requirements.txt  Dockerfile  README.md
```

## 5 Folder Responsibilities

**api/** — HTTP routes, request validation, auth, response formatting. *No business logic*; requests are forwarded to the planner.

**core/** — app-wide config, constants, logging, security, dependency wiring, loaded at startup.

**models/** — every Pydantic model (`ChatRequest`, `ChatResponse`, `ToolCall`, `Merchant`, `Offer`, `User`, `MemoryState`) defining the contract between components.

**services/** — reusable external-communication layers: Firestore, embeddings, LLM, Google Places, speech, merchant data, recommendations.

**planner/** — the most important module. Decides whether a request should generate text, call a tool, retrieve documents, or combine several of these. May later evolve into a graph-based orchestrator; the MVP can be a rule-based dispatcher augmented by the LLM.

**rag/** — retrieval-augmented generation: document loader, embedding generator, vector search, retriever, re-ranker, knowledge base. Never talks to Flutter directly.

**llm/** — interface to the local model (initially **Qwen3** via **Ollama**); future models should be swappable without touching the rest of the backend.

**nlp/** — pre-LLM language understanding: intent classification, entity extraction, language detection, text preprocessing. Longer term, spaCy and sentence-transformers may work alongside the LLM.

**memory/** — conversation state, user preferences, session data, short-term context, kept independent of the LLM itself.

**prompts/** — every prompt template (`planner_prompt.txt`, `merchant_prompt.txt`, `chat_prompt.txt`, `tool_prompt.txt`, `rag_prompt.txt`), never hardcoded in Python, so prompts evolve independently of application logic.

**utils/** — generic helpers: JSON formatting, validators, string/time helpers, distance calculations. No AI logic lives here.

## 6 Development Order

A phased build keeps the backend functional at every stage:

| # | Component | Goal |
|---|---|---|
| 1 | FastAPI Skeleton | Project structure, dependencies, health endpoint |
| 2 | API Models | Pydantic request/response schemas |
| 3 | Planner | Basic dispatcher, placeholder logic |
| 4 | Tool Framework | Abstract tool interface & stubs |
| 5 | LLM Service | Local model (Qwen via Ollama) behind a clean interface |
| 6 | RAG Layer | Indexing, retrieval, prompt augmentation |
| 7 | Memory | Conversation state & preferences |
| 8 | Service Integrations | Firestore, Google Places, merchant & recommendation APIs |
| 9 | Testing & Observability | Logging, metrics, automated tests |